# CUDA Crash Course

A Fast Self-Study CUDA Note with C and CUDA Side-by-Side

This note was organized during my PhD years with a very simple goal: to help a beginner learn CUDA by self-study as quickly as possible. For that reason, I kept it short, direct, and practical. Every major example comes in two versions, one in C and one in CUDA, so the reader can compare them quickly without getting lost. In this updated version, the code has been cleaned up and adjusted to better fit modern CUDA architecture. In a way, this note is also a small memo from my early research days.

Chun-Yu Lu

English LaTeX edition

March 27, 2026

# Outline

This page is the reading map for the paired examples and extensions. Each entry below jumps to the matching section, and each example section links back to the same line here.

## Core note


- Preface
- Recommended Learning Environment: Google Colab with a Free T4 GPU
- The Seven Essential Moves
- Common CUDA Rules That Help Beginners Avoid Trouble


## Examples and extensions


- Example 1: Writing a Message
- Example 2: Copying an Array
- Example 3: Filling an Array with Its Index
- Example 4: Vector Addition
- Example 5: Three-Point Smoothing
- Example 6: Unified Memory as a Teaching Shortcut
- A Paired Template for New Projects
- Extension 1: Streams and Asynchronous Copy
- Extension 2: Memory Coalescing
- Extension 3: Error Handling as a Real Habit

# 1. Preface

This note was organized during my PhD years with a very simple goal: to help a beginner learn CUDA by self-study as quickly as possible. For that reason, I kept it short, direct, and practical. Every major example comes in two versions, one in C and one in CUDA, so the reader can compare them quickly without getting lost. In this updated version, the code has been cleaned up and adjusted to better fit modern CUDA architecture. In a way, this note is also a small memo from my early research days.

This is a fast entry note, not a full CUDA manual. The point is to make the core model visible and usable: host versus device, kernel launch, indexing, memory movement, synchronization, and the first layer of performance thinking.

# 2. Recommended Learning Environment: Google Colab with a Free T4 GPU

If the goal is simply to learn CUDA and start running kernels, **Google Colab is a very sensible place to begin**. No driver wrestling, no local toolkit drama, and no losing half a day because one path variable is in a bad mood. You get a real NVIDIA GPU right away. In the setup shown here, the runtime is `Python 3` with a **T4 GPU**, and that is already plenty for every example in this document.

**Practical recommendation for readers:** Use Colab with a GPU runtime, and when a **T4** shows up, just take it and start. It is more than enough for learning the basics covered here: host/device memory, kernel launch syntax, indexing, explicit memory copies, Unified Memory, and small shared-memory kernels.

## 2.1 A first Colab check

The first command the reader should run is:

Listing 1: Check the GPU in Colab

```
!nvidia-smi
```

A representative output from the user's session looks like this:

Listing 2: Representative T4 runtime information from Colab

```
NVIDIA-SMI 580.82.07 Driver Version: 580.82.07 CUDA Version: 13.0
GPU Name: Tesla T4
Memory-Usage: 0 MiB / 15360 MiB
No running processes found
```

That tiny output already tells you quite a lot if you know what to look for.

## 2.2 What the memory line means

The line

`0 MiB / 15360 MiB`

means that the GPU currently has about **15360 MiB of device memory in total**, which is about **15 GiB of VRAM**. At that moment, none of it is in active use by a running process.

That is good news for a beginner. These examples are tiny compared with 15 GiB, so memory is not the boss fight today. You can pay attention to the CUDA model itself instead of worrying that one small array already ate the machine.

| Quantity | Meaning |
|---|---|
| `15360 MiB` | Total GPU global memory visible to the runtime |
| `0 MiB` | Currently used GPU memory at the time of the check |
| `No running processes found` | No active job is holding GPU memory yet |

## 2.3 The memory concepts a beginner should know immediately

Right at the start, it helps to separate **four different memory ideas**:

1. **Host RAM**: ordinary CPU-side memory used by C, Python, NumPy, and normal system programs.
2. **GPU global memory**: the main device memory used by `cudaMalloc` and usually reported by `nvidia-smi`.
3. **Shared memory**: small, fast, block-local on-chip memory used for thread cooperation.
4. **Registers**: the fastest per-thread storage, managed primarily by the compiler.

The main correction beginners need is this: **the 15360 MiB reported by `nvidia-smi` is device global memory**. It is *not* shared memory, and it is *not* registers. Shared memory and registers are much smaller on-chip resources. They are controlled per kernel launch and per streaming multiprocessor, so they do not show up as that big VRAM number in `nvidia-smi`.

## 2.4 High-RAM in Colab is not GPU RAM

In Colab, the **High-RAM** switch affects the host-side memory available to the notebook runtime. It does *not* give your T4 extra VRAM. This matters because beginners very often mix up the following two things:

- **RAM for Python / CPU data structures**, and
- **VRAM for GPU-resident arrays and kernels**.

So if a CUDA program fails on `cudaMalloc`, the problem is usually **device memory**, not notebook RAM. Turning on High-RAM will not magically persuade the GPU to grow new memory chips overnight.

## 2.5 Memory-size intuition for readers

It is worth building a rough byte-level feeling early on:

| Object | Approximate size |
|---|---|
| **1 `float`** | 4 bytes |
| **1 million `floats`** | about 3.8 MiB |
| **100 million `floats`** | about 381.5 MiB |
| **Three arrays of 100 million `floats`** | about 1.12 GiB |

So even a free T4-class Colab runtime is already enough for serious learning exercises. The examples in this note are far below that scale, by design.

### 2.6 One more technical note: the CUDA version line

When `nvidia-smi` shows a line such as `CUDA Version: 13.0`, read that as the **driver-supported CUDA capability reported by the system**. Do not over-read it as the exact `nvcc` toolkit version inside the notebook. For our purpose, the practical message is simpler: the runtime is modern enough to compile and run ordinary CUDA examples without trouble.

### 2.7 Compile flag for a T4 in Colab

A T4 GPU corresponds to compute capability 7.5, so a clean Colab habit is to compile the CUDA examples with `-arch=sm_75`. That keeps the command explicit and avoids the warning noise that sometimes appears when the default target list is broader than necessary.

Listing 3: A clean Colab compile command for a T4 runtime

```
!nvcc code/cuda/04_vector_add_gpu.cu -O2 -arch=sm_75 -o ex04_gpu
!./ex04_gpu
```

## 3. The Seven Essential Moves

### 3.1 Move 1: Host and Device

CUDA starts by separating two execution worlds:

- **Host**: the CPU and the normal C/C++ program running on the system.
- **Device**: the GPU, where kernels execute.

The host and device have different execution contexts and, in the usual explicit-memory model, different memory spaces. The host allocates ordinary system memory with `malloc`, `new`, or container-managed storage. The device allocates GPU memory with CUDA runtime calls such as `cudaMalloc`. Data moves between them with calls such as `cudaMemcpy`.

The basic work pattern is:

1. prepare input data on the host,
2. allocate device memory,
3. move input data to the device,
4. launch one or more kernels,
5. move the results back to the host,
6. free GPU memory.

For performance, it is usually better to keep data on the device across several kernels instead of bouncing it back and forth across the CPU–GPU boundary. PCIe is useful, but it is not there to be abused for fun.

**Practical note.** It is fine to say "host = CPU side" and "device = GPU side" at the start. Just remember that CUDA is really separating *execution domains* and *memory spaces*, not only labeling pieces of hardware.

## 3.2 Move 2: Essential Runtime API Calls

On day one, you do not need twenty APIs. A very small set of runtime calls is enough to get real work done.

| API | Role |
|---|---|
| `cudaMalloc` | Allocate device memory |
| `cudaFree` | Free device memory |
| `cudaMemcpy` | Copy between host and device |
| `cudaGetLastError` | Check the most recent launch error |
| `cudaGetErrorString` | Convert an error code to readable text |
| `cudaDeviceSynchronize` | Wait until previously issued device work is complete |

A modern reader should also know that CUDA supports **Unified Memory** via `cudaMallocManaged`. Unified Memory cuts away a lot of boilerplate and is great for learning or quick prototyping, but explicit memory management is still worth learning because it shows the real dataflow of a GPU program.

### 3.2.1 A minimal error-checking pattern

Listing 4: A minimal runtime error-checking macro

```
#include <cstdio>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
    if (e != cudaSuccess) { \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                    __FILE__, __LINE__, cudaGetErrorString(e)); \
        std::exit(EXIT_FAILURE); \
    } \
} while (0)
```

If you skip error checking, beginners often spend an hour blaming their math while the kernel launch actually failed three lines earlier. That is one of the quickest ways to waste a perfectly good afternoon.

## 3.3 Move 3: Functions and Kernel Launch

A host function looks like an ordinary C/C++ function. A device kernel is marked with `__global__` and launched with the chevron syntax:

Listing 5: Kernel definition and launch syntax

```
__global__ void saxpy(float* y, const float* x, float a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a * x[i] + y[i];
}

// host code
int threads = 256;
int blocks = (n + threads - 1) / threads;
saxpy<<<blocks, threads>>>(d_y, d_x, a, n);
CUDA_CHECK(cudaGetLastError());
```

```
CUDA_CHECK(cudaDeviceSynchronize());
```

The key points are simple:

- a kernel returns `void`,
- work decomposition is encoded by thread and block indices,
- pointers passed to kernels usually refer to device-accessible memory,
- launch failure must be checked explicitly.

## 3.4 Move 4: Grid, Block, and Thread

CUDA's execution hierarchy is:

$$\text{grid} \to \text{blocks} \to \text{threads}.$$

Each thread sees built-in variables such as:

- `threadIdx`: thread index within a block,
- `blockIdx`: block index within a grid,
- `blockDim`: block shape,
- `gridDim`: grid shape.

The one formula every beginner should know early is the one-dimensional global index:

$$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}.$$

For multidimensional layouts, the same principle applies: compute a unique logical coordinate from block and thread coordinates, then map it to your data structure.

### 3.4.1 One-dimensional indexing

Listing 6: A simple 1D indexing example

```
__global__ void write_indices(int* out) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    out[i] = i;
}
```

### 3.4.2 Three-dimensional flattening

Suppose each thread lives in a 3D block and each block lives in a 3D grid. A standard flattening pattern is:

$$\begin{aligned} b &= blockIdx.x + gridDim.x\,(blockIdx.y + gridDim.y \cdot blockIdx.z)\,, \\ t &= threadIdx.x + blockDim.x\,(threadIdx.y + blockDim.y \cdot threadIdx.z)\,, \\ i &= b \cdot (blockDim.xblockDim.yblockDim.z) + t. \end{aligned}$$

There is no CUDA magic hiding here. It is just row-major linearization of a multidimensional index space.

**Practical note on launch geometry.** Use `dim3` naturally. Think of the grid and the block as genuine 1D, 2D, or 3D index spaces, and only worry about exact hardware limits when the program actually needs them.

## 3.5 Move 5: Memory Hierarchy

A clean first-pass memory hierarchy is shown below.

| Memory space | Typical use | Main idea |
|---|---|---|
| **Host memory** | **CPU-side storage** | Ordinary system memory |
| **Global memory** | **Main GPU data storage** | Large, high-latency, visible to all threads |
| **Shared memory** | **Block-local cooperation** | Fast on-chip storage shared by threads in one block |
| **Registers** | **Per-thread temporaries** | Fastest storage; heavily resource-limited |
| **Constant memory** | **Read-only broadcast data** | Useful when many threads read the same values |
| **Local memory** | **Compiler spill or large per-thread arrays** | Physically backed by device memory despite the name |

For beginners, the right place to start is still host memory, device memory, shared memory, and registers. Two extra cautions matter immediately:

1. **"Local" memory is not on-chip.** It often means spilled or compiler-managed storage backed by device memory.
2. **Register pressure matters.** A kernel can become slower if excessive registers reduce occupancy or force spills.

### 3.5.1 When shared memory is worth using

Shared memory helps when threads in the same block reuse nearby data or need to exchange partial results. Good examples are tiled matrix operations, stencil kernels, reductions, and transposes. If there is no reuse and no cooperation, shared memory is often just extra ceremony.

## 3.6 Move 6: Synchronization

At the beginning, two synchronization ideas are enough:

1. **Block-level barrier inside a kernel**: `__syncthreads()`.
2. **Host waits for the GPU**: `cudaDeviceSynchronize()`.

### 3.6.1 Inside a block

`__syncthreads()` is a barrier for all threads in the same block. It is typically used after threads cooperatively load shared memory and before they consume the shared data.

### 3.6.2 On the host

`cudaDeviceSynchronize()` forces the host to wait until the device has completed all prior work in the default execution context. It is useful for debugging, timing, and for those moments when you want the host to stop being overly optimistic.

**Practical note.** When the host really needs to wait for the GPU, use `cudaDeviceSynchronize()`. Do not sprinkle it everywhere for no reason, because that just turns parallel work into a traffic jam.

## 3.7 Move 7: Performance Path

Coalesced access is still one of the decisive performance ideas. The beginner version of the rule should just be stated clearly and without drama.

**Modern performance rule for beginners:** Try to make neighboring threads access neighboring memory locations, keep control flow simple, avoid unnecessary host-device transfers, and use shared memory only when it clearly earns its keep through reuse or communication.

Early CUDA documentation often described memory coalescing with half-warps and very strict alignment rules. That was the right story for first-generation devices. Modern architectures are more forgiving and have better cache behavior, but the big lesson survives even though the hardware details changed:

- contiguous accesses are usually better than scattered accesses,
- regular patterns are easier for the hardware to serve efficiently,
- shared memory tiling should be justified by reuse, not ritual,
- performance claims should always be verified by measurement.

# 4. Common CUDA Rules That Help Beginners Avoid Trouble

## 4.1 A compact rule table

| What you may still see online | Recommended practice here | Comment |
|---|---|---|
| `#include <cuda.h>` in runtime examples | `#include <cuda_runtime.h>` | Prefer the runtime header for runtime-API examples. |
| `cudaThreadSynchronize()` | `cudaDeviceSynchronize()` | The former is deprecated. |
| **"Max block size is 512"** | **"Many current devices allow up to 1024 threads per block"** | Actual limits should be queried from the device when writing portable software. |
| **Strict half-warp coalescing rules as the primary story** | **Neighboring threads should read neighboring locations** | The main idea remains valid; the micro-rules are architecture-specific and not the main point here. |
| **Texture references as the first optimization story** | **Use ordinary global memory first; consider read-only data paths, shared-memory tiling, or texture objects only for the right access pattern** | Texture objects are specialized tools, not the front door. |
| **Manual copies in every pedagogical example** | **Teach both explicit copies and Unified Memory** | Unified Memory is excellent for small modern examples, but explicit copies still teach the model better. |

## 4.2 Texture objects: use them only when the access pattern really asks for them

Texture objects still exist, and they can be useful. But they should not be the first hammer you hand to a beginner.

Today, the first optimization questions should usually be:

1. Is the memory access pattern regular?
2. Are neighboring threads reading neighboring locations?
3. Is there data reuse worth staging in shared memory?
4. Is the data read-only?
5. Does the kernel bottleneck on bandwidth, arithmetic throughput, or occupancy?

Texture objects still exist and are still useful for certain read patterns, especially when their addressing and caching semantics really match the problem. But now they are a specialized tool, not the front door for memory optimization.

## 4.3 Warp-level reasoning: useful, but teach it at the right time

Warp behavior matters. Modern CUDA also makes warp-level programming more visible through intrinsics and libraries, so this topic should be taught with precise synchronization semantics instead of hand-waving and wishful thinking.

A sensible learning order is:

1. first learn block-level programming,
2. then learn how warps execute,
3. then introduce warp intrinsics such as `__syncwarp()` and warp shuffles,
4. only after that move to deeper occupancy and latency-hiding analysis.

# 5. Modern Code Examples: C Version vs CUDA Version

From this point on, every complete example is written in a **paired format** and tied to an actual source file in the package zip. So the reader does not need to copy code by hand like it is 2010. The LaTeX note and the code archive are synchronized.

## 5.1 Package structure used in this document

| Folder | Purpose |
|---|---|
| `code/c/` | CPU-only reference programs in plain C |
| `code/cuda/` | CUDA versions of the same programs |
| `README_COLAB_T4.md` | Colab usage notes, compile commands, and memory reminders |
| `RUN_ALL_COLAB.md` | One-cell commands to compile and run the full example set in Colab |

## 5.2 Example 1: Writing a Message

**Files for this example**

- C version: `code/c/01_write_message_cpu.c`
- CUDA version: `code/cuda/01_write_message_gpu.cu`

**Colab compile commands**

- `!gcc code/c/01_write_message_cpu.c -O2 -o ex01_cpu`
- `!nvcc code/cuda/01_write_message_gpu.cu -O2 -arch=sm_75 -o ex01_gpu`

### 5.2.1 C version

Listing 7: Filename: `code/c/01_write_message_cpu.c` — C version: write a message on the CPU

```c
#include <stdio.h>
#include <string.h>

void write_message(char* s) {
    const char msg[] = "hello C";
    int k = 0;
    while (msg[k] != '\0') {
        s[k] = msg[k];
        ++k;
    }
    s[k] = '\0';
}
```

```

int main(void) {
    char s[64] = {0};
    write_message(s);
    printf("%s\n", s);
    return 0;
}
```

### 5.2.2 CUDA version

Listing 8: Filename: `code/cuda/01_write_message_gpu.cu` — CUDA version: write the same message on the GPU and copy it back

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
    if (e != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
               __FILE__, __LINE__, cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

__global__ void write_message_kernel(char* s) {
    const char msg[] = "hello CUDA";
    int k = 0;
    while (msg[k] != '\0') {
        s[k] = msg[k];
        ++k;
    }
    s[k] = '\0';
}

int main(void) {
    char* d_s = NULL;
    char h_s[64] = {0};

    CUDA_CHECK(cudaMalloc((void**)&d_s, sizeof(h_s)));

    write_message_kernel<<<1,1>>>(d_s);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_s, d_s, sizeof(h_s), cudaMemcpyDeviceToHost));
    printf("%s\n", h_s);

    CUDA_CHECK(cudaFree(d_s));
    return 0;
}
```

The computation is still trivial. The real lesson is structural: the C version calls a normal function directly, while the CUDA version must allocate GPU memory, launch a kernel, wait for completion, and copy the result back.

Back to the outline page

## 5.3 Example 2: Copying an Array

**Files for this example**

- C version: `code/c/02_array_copy_cpu.c`
- CUDA version: `code/cuda/02_array_copy_gpu.cu`

### 5.3.1 C version

Listing 9: Filename: `code/c/02_array_copy_cpu.c` — C version: copy one array into another on the CPU

```c
#include <stdio.h>

int main(void) {
    enum { N = 100 };
    int a[N], b[N];
    int i;
    int ok = 1;

    for (i = 0; i < N; ++i) {
        a[i] = i;
        b[i] = 0;
    }

    for (i = 0; i < N; ++i) {
        b[i] = a[i];
    }

    for (i = 0; i < N; ++i) {
        if (a[i] != b[i]) {
            ok = 0;
            break;
        }
    }

    printf("check a == b ? %s\n", ok ? "pass" : "wrong");
    return 0;
}
```

### 5.3.2 CUDA version

Listing 10: Filename: `code/cuda/02_array_copy_gpu.cu` — CUDA version: copy host data to the device and back

```c
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
    if (e != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
```

```
    } \
} while (0)

int main(void) {
    enum { N = 100 };
    int a[N], b[N];
    int* d = NULL;
    int i;
    int ok = 1;

    for (i = 0; i < N; ++i) {
        a[i] = i;
        b[i] = 0;
    }

    CUDA_CHECK(cudaMalloc((void**)&d, N * sizeof(int)));
    CUDA_CHECK(cudaMemcpy(d, a, N * sizeof(int), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(b, d, N * sizeof(int), cudaMemcpyDeviceToHost));

    for (i = 0; i < N; ++i) {
        if (a[i] != b[i]) {
            ok = 0;
            break;
        }
    }

    printf("check a == b ? %s\n", ok ? "pass" : "wrong");
    CUDA_CHECK(cudaFree(d));
    return 0;
}
```

In the C version, the copy is just a direct CPU-side loop. In the CUDA version, the interesting part is not parallelism yet; it is **host-device data movement**. This is the smallest example that makes the memory boundary impossible to ignore.

Back to the outline page

## 5.4 Example 3: Filling an Array with Its Index

**Files for this example**

- C version: `code/c/03_write_index_cpu.c`
- CUDA version: `code/cuda/03_write_index_gpu.cu`

### 5.4.1 C version

Listing 11: Filename: `code/c/03_write_index_cpu.c` — C version: one serial loop writes the index

```
#include <stdio.h>

int main(void) {
    enum { N = 12 };
    int out[N];
    int i;

```

```
    for (i = 0; i < N; ++i) {
        out[i] = i;
    }

    for (i = 0; i < N; ++i) {
        printf("out[%d] = %d\n", i, out[i]);
    }
    return 0;
}
```

### 5.4.2 CUDA version

Listing 12: Filename: `code/cuda/03_write_index_gpu.cu` — CUDA version: each thread writes one global index

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
    if (e != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

__global__ void write_index_kernel(int* out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = i;
    }
}

int main(void) {
    enum { N = 12 };
    int h_out[N];
    int* d_out = NULL;
    int i;
    int threads = 4;
    int blocks = (N + threads - 1) / threads;

    CUDA_CHECK(cudaMalloc((void**)&d_out, N * sizeof(int)));

    write_index_kernel<<<blocks, threads>>>(d_out, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_out, d_out, N * sizeof(int), cudaMemcpyDeviceToHost));

    for (i = 0; i < N; ++i) {
        printf("out[%d] = %d\n", i, h_out[i]);
    }

    CUDA_CHECK(cudaFree(d_out));
    return 0;
```

```
}
```

This is the first example where the **loop index itself becomes the bridge** between C and CUDA. In C, one CPU loop variable walks through the array. In CUDA, many threads each rebuild the one index they own.

Back to the outline page

## 5.5 Example 4: Vector Addition

**Files for this example**

- C version: `code/c/04_vector_add_cpu.c`
- CUDA version: `code/cuda/04_vector_add_gpu.cu`

### 5.5.1 C version

Listing 13: Filename: `code/c/04_vector_add_cpu.c` — C version: serial vector addition on the CPU

```c
#include <stdio.h>

int main(void) {
    enum { N = 8 };
    float a[N], b[N], c[N];
    int i;

    for (i = 0; i < N; ++i) {
        a[i] = (float)i;
        b[i] = 100.0f + (float)i;
    }

    for (i = 0; i < N; ++i) {
        c[i] = a[i] + b[i];
    }

    for (i = 0; i < N; ++i) {
        printf("c[%d] = %.1f\n", i, c[i]);
    }
    return 0;
}
```

### 5.5.2 CUDA version

Listing 14: Filename: `code/cuda/04_vector_add_gpu.cu` — CUDA version: one thread computes one element of `c = a + b`

```c
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
    if (e != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
```

```
                __FILE__, __LINE__, cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

__global__ void vec_add_kernel(float* c, const float* a, const float* b, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main(void) {
    enum { N = 8 };
    float h_a[N], h_b[N], h_c[N];
    float *d_a = NULL, *d_b = NULL, *d_c = NULL;
    int i;
    int threads = 256;
    int blocks = (N + threads - 1) / threads;

    for (i = 0; i < N; ++i) {
        h_a[i] = (float)i;
        h_b[i] = 100.0f + (float)i;
    }

    CUDA_CHECK(cudaMalloc((void**)&d_a, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&d_b, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&d_c, N * sizeof(float)));

    CUDA_CHECK(cudaMemcpy(d_a, h_a, N * sizeof(float), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, N * sizeof(float), cudaMemcpyHostToDevice));

    vec_add_kernel<<<blocks, threads>>>(d_c, d_a, d_b, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_c, d_c, N * sizeof(float), cudaMemcpyDeviceToHost));

    for (i = 0; i < N; ++i) {
        printf("c[%d] = %.1f\n", i, h_c[i]);
    }

    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    return 0;
}
```

This pair is the cleanest first parallel-programming example. The arithmetic does not change at all. The only conceptual jump is that the serial loop body

$$c[i] = a[i] + b[i]$$

becomes the work assigned to one GPU thread. Same math, different workforce.

Back to the outline page

## 5.6 Example 5: Three-Point Smoothing

**Files for this example**

- C version: `code/c/05_three_point_smoothing_cpu.c`
- CUDA version: `code/cuda/05_three_point_smoothing_gpu.cu`

### 5.6.1 C version

Listing 15: Filename: `code/c/05_three_point_smoothing_cpu.c` — C version: three-point smoother on the CPU

```c
#include <stdio.h>

void smooth_cpu(float* out, const float* in, int n) {
    int i;
    if (n <= 0) return;

    out[0] = 0.25f * (2.0f * in[0] + (n > 1 ? in[1] : 0.0f));
    for (i = 1; i < n - 1; ++i) {
        out[i] = 0.25f * (in[i - 1] + 2.0f * in[i] + in[i + 1]);
    }
    if (n > 1) {
        out[n - 1] = 0.25f * (in[n - 2] + 2.0f * in[n - 1]);
    }
}

int main(void) {
    enum { N = 16 };
    float in[N], out[N];
    int i;

    for (i = 0; i < N; ++i) {
        in[i] = (float)(i % 7);
    }

    smooth_cpu(out, in, N);

    for (i = 0; i < N; ++i) {
        printf("out[%d] = %.2f\n", i, out[i]);
    }
    return 0;
}
```

### 5.6.2 CUDA version

Listing 16: Filename: `code/cuda/05_three_point_smoothing_gpu.cu` — CUDA version: the same smoother using shared memory

```c
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
```

```
    if (e != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

#define BLOCK_SIZE 256

__global__ void smooth_shared_kernel(float* out, const float* in, int n) {
    __shared__ float tile[BLOCK_SIZE + 2];

    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    int tid = threadIdx.x;

    tile[tid + 1] = (gid < n) ? in[gid] : 0.0f;

    if (tid == 0) {
        int left = gid - 1;
        tile[0] = (left >= 0) ? in[left] : 0.0f;
    }

    if (tid == blockDim.x - 1) {
        int right = gid + 1;
        tile[BLOCK_SIZE + 1] = (right < n) ? in[right] : 0.0f;
    }

    __syncthreads();

    if (gid < n) {
        float left = tile[tid];
        float center = tile[tid + 1];
        float right = tile[tid + 2];
        out[gid] = 0.25f * (left + 2.0f * center + right);
    }
}

int main(void) {
    enum { N = 16 };
    float h_in[N], h_out[N];
    float *d_in = NULL, *d_out = NULL;
    int i;
    int threads = BLOCK_SIZE;
    int blocks = (N + threads - 1) / threads;

    for (i = 0; i < N; ++i) {
        h_in[i] = (float)(i % 7);
    }

    CUDA_CHECK(cudaMalloc((void**)&d_in, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&d_out, N * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice));

    smooth_shared_kernel<<<blocks, threads>>>(d_out, d_in, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost));
```

```

    for (i = 0; i < N; ++i) {
        printf("out[%d] = %.2f\n", i, h_out[i]);
    }

    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));
    return 0;
}
```

This is the first pair where the CUDA version is not just a loop-to-thread translation. It introduces a **new storage level**, shared memory, because neighboring threads reuse neighboring data. This is exactly the kind of case where shared memory is justified instead of being added just because someone felt guilty not using it.

Back to the outline page

## 5.7 Example 6: Unified Memory as a Teaching Shortcut

**Files for this example**

- C version: `code/c/06_unified_memory_scale_cpu.c`
- CUDA version: `code/cuda/06_unified_memory_scale_gpu.cu`

### 5.7.1 C version

Listing 17: Filename: `code/c/06_unified_memory_scale_cpu.c` — C version: scale an array in place on the CPU

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    enum { N = 8 };
    float* data = (float*)malloc(N * sizeof(float));
    int i;

    for (i = 0; i < N; ++i) {
        data[i] = (float)i;
    }

    for (i = 0; i < N; ++i) {
        data[i] = 2.0f * data[i];
    }

    for (i = 0; i < N; ++i) {
        printf("data[%d] = %.1f\n", i, data[i]);
    }

    free(data);
    return 0;
}
```

### 5.7.2 CUDA version

Listing 18: Filename: `code/cuda/06_unified_memory_scale_gpu.cu` — CUDA version: the same operation using Unified Memory

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
    if (e != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

__global__ void scale_kernel(float* data, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        data[i] = 2.0f * data[i];
    }
}

int main(void) {
    enum { N = 8 };
    float* data = NULL;
    int i;
    int threads = 256;
    int blocks = (N + threads - 1) / threads;

    CUDA_CHECK(cudaMallocManaged((void**)&data, N * sizeof(float)));

    for (i = 0; i < N; ++i) {
        data[i] = (float)i;
    }

    scale_kernel<<<blocks, threads>>>(data, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    for (i = 0; i < N; ++i) {
        printf("data[%d] = %.1f\n", i, data[i]);
    }

    CUDA_CHECK(cudaFree(data));
    return 0;
}
```

Unified Memory is useful because it removes a lot of boilerplate. A student can focus on **parallel decomposition** first and postpone some of the copy mechanics. But it should still be presented as a convenience layer, not as an excuse to forget the host-device boundary exists.

Back to the outline page

# 6. A Paired Template for New Projects

One of the most useful things in this note is the final side-by-side template. Many readers understand each example in isolation but still freeze when they open a blank file. The paired template below shows the minimum structural jump from C to CUDA.

**Files for the reusable template**

- C template: `code/c/07_transform_template_cpu.c`
- CUDA template: `code/cuda/07_transform_template_gpu.cu`

## 6.1 C template

Listing 19: Filename: `code/c/07_transform_template_cpu.c` — C template: serial array transform

```c
#include <stdio.h>
#include <stdlib.h>

void transform(float* out, const float* in, int n) {
    int i;
    for (i = 0; i < n; ++i) {
        out[i] = 2.0f * in[i];
    }
}

int main(void) {
    const int N = 1 << 20;
    float* in = (float*)malloc(N * sizeof(float));
    float* out = (float*)malloc(N * sizeof(float));
    int i;

    for (i = 0; i < N; ++i) {
        in[i] = 1.0f;
    }

    transform(out, in, N);
    printf("out[0] = %f\n", out[0]);

    free(in);
    free(out);
    return 0;
}
```

## 6.2 CUDA template

Listing 20: Filename: `code/cuda/07_transform_template_gpu.cu` — CUDA template: parallel array transform

```c
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t e = (call); \
```

```
    if (e != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

__global__ void transform_kernel(float* out, const float* in, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = 2.0f * in[i];
    }
}

int main(void) {
    const int N = 1 << 20;
    float *h_in = NULL, *h_out = NULL;
    float *d_in = NULL, *d_out = NULL;
    int i;
    int threads = 256;
    int blocks = (N + threads - 1) / threads;

    h_in = (float*)malloc(N * sizeof(float));
    h_out = (float*)malloc(N * sizeof(float));

    for (i = 0; i < N; ++i) {
        h_in[i] = 1.0f;
    }

    CUDA_CHECK(cudaMalloc((void**)&d_in, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&d_out, N * sizeof(float)));

    CUDA_CHECK(cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice));

    transform_kernel<<<blocks, threads>>>(d_out, d_in, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost));

    printf("out[0] = %f\n", h_out[0]);

    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));
    free(h_in);
    free(h_out);
    return 0;
}
```

**The structural upgrade from C to CUDA can be memorized in one sentence:** Take the serial loop body, turn it into a kernel body guarded by a global index, then add device allocation, host-device transfers, a launch configuration, and error checking. That is the whole trick, at least for the first round.

Back to the outline page

# 7. Recommended Next Topics After This Crash Course

Once the reader is comfortable with the material above, including the extension section, the next topics should be:

1. events and timing,
2. constant memory and read-only data paths,
3. reductions and prefix-style block cooperation,
4. warp intrinsics and `__syncwarp()`,
5. occupancy, register pressure, and launch configuration,
6. profiling with Nsight tools,
7. device query and build flow habits,
8. texture objects and surface objects only when the access pattern truly calls for them.

# 8. Extension: Three More Practical Moves

To keep the note moving, I add only a few things at the end, not a whole truckload. So here I pick **three extensions** that fit this note well:

1. streams with asynchronous copy,
2. memory coalescing,
3. deeper error-checking patterns.

I did *not* choose everything on the longer wish list. Topics like occupancy, Nsight, or full profiling are important, but they belong to the second training round. First learn to write kernels that are structurally right. Then make them fast. Otherwise students start benchmarking chaos, which is a surprisingly popular hobby.

## 8.1 Extension 1: Streams and Asynchronous Copy

A plain C program usually does one chunk, then the next chunk, then the next. Very honest. Very obedient. CUDA gives you another option: **while one chunk is being copied, another chunk can already be computing**. That is where streams become useful.

**Files for this extension**

- C version: `code/c/08_stream_chunk_pipeline_cpu.c`
- CUDA version: `code/cuda/08_stream_chunk_pipeline_gpu.cu`

### 8.1.1 C version

Listing 21: Filename: `code/c/08_stream_chunk_pipeline_cpu.c` — C version: process one chunk at a time on the CPU

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

static void saxpy_chunk(float *data, float a, int n)
{
    for (int i = 0; i < n; ++i) {
        data[i] = a * data[i] + 1.0f;
    }
}

int main(void)
{
    const int N = 1 << 20;
    const int CHUNK = 1 << 16;
    const float a = 2.0f;

    float *in = (float *)malloc((size_t)N * sizeof(float));
    float *stage = (float *)malloc((size_t)CHUNK * sizeof(float));
    float *out = (float *)malloc((size_t)N * sizeof(float));

    if (!in || !stage || !out) {
        fprintf(stderr, "CPU allocation failed\n");
        free(in);
        free(stage);
        free(out);
        return EXIT_FAILURE;
    }

    for (int i = 0; i < N; ++i) {
        in[i] = (float)i;
    }

    for (int offset = 0; offset < N; offset += CHUNK) {
        const int count = (offset + CHUNK <= N) ? CHUNK : (N - offset);
        memcpy(stage, in + offset, (size_t)count * sizeof(float));
        saxpy_chunk(stage, a, count);
        memcpy(out + offset, stage, (size_t)count * sizeof(float));
    }

    printf("CPU chunk pipeline: out[0]=%.1f out[N-1]=%.1f\n", out[0], out[N - 1]);

    free(in);
    free(stage);
    free(out);
    return EXIT_SUCCESS;
}
```

### 8.1.2 CUDA version

Listing 22: Filename: `code/cuda/08_stream_chunk_pipeline_gpu.cu` — CUDA version: overlap chunk copies and kernel work with streams

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) \
    do { \
        cudaError_t err = (call); \
```

```
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
                    cudaGetErrorString(err)); \
            return EXIT_FAILURE; \
        } \
    } while (0)

__global__ void saxpy_kernel(float *out, const float *in, float a, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = a * in[i] + 1.0f;
    }
}

int main(void)
{
    const int N = 1 << 20;
    const int CHUNK = 1 << 16;
    const int NSTREAMS = 2;
    const float a = 2.0f;

    float *h_in = NULL;
    float *h_out = NULL;
    float *d_in[NSTREAMS] = {NULL, NULL};
    float *d_out[NSTREAMS] = {NULL, NULL};
    cudaStream_t streams[NSTREAMS];

    CHECK_CUDA(cudaMallocHost((void **)&h_in, (size_t)N * sizeof(float)));
    CHECK_CUDA(cudaMallocHost((void **)&h_out, (size_t)N * sizeof(float)));

    for (int s = 0; s < NSTREAMS; ++s) {
        CHECK_CUDA(cudaStreamCreate(&streams[s]));
        CHECK_CUDA(cudaMalloc((void **)&d_in[s], (size_t)CHUNK * sizeof(float)));
        CHECK_CUDA(cudaMalloc((void **)&d_out[s], (size_t)CHUNK * sizeof(float)));
    }

    for (int i = 0; i < N; ++i) {
        h_in[i] = (float)i;
    }

    for (int offset = 0; offset < N; offset += CHUNK) {
        int s = (offset / CHUNK) % NSTREAMS;
        int count = (offset + CHUNK <= N) ? CHUNK : (N - offset);
        size_t bytes = (size_t)count * sizeof(float);
        int threads = 256;
        int blocks = (count + threads - 1) / threads;

        CHECK_CUDA(cudaMemcpyAsync(d_in[s], h_in + offset, bytes,
                                   cudaMemcpyHostToDevice, streams[s]));
        saxpy_kernel<<<blocks, threads, 0, streams[s]>>>(d_out[s], d_in[s], a, count);
        CHECK_CUDA(cudaGetLastError());
        CHECK_CUDA(cudaMemcpyAsync(h_out + offset, d_out[s], bytes,
                                   cudaMemcpyDeviceToHost, streams[s]));
    }

    for (int s = 0; s < NSTREAMS; ++s) {
        CHECK_CUDA(cudaStreamSynchronize(streams[s]));
```

```
    }

    printf("GPU stream pipeline: out[0]=%.1f out[N-1]=%.1f\n", h_out[0], h_out[N - 1])
        ;

    for (int s = 0; s < NSTREAMS; ++s) {
        cudaFree(d_in[s]);
        cudaFree(d_out[s]);
        cudaStreamDestroy(streams[s]);
    }
    cudaFreeHost(h_in);
    cudaFreeHost(h_out);
    return EXIT_SUCCESS;
}
```

The C version is not wrong. It is just **serial chunking**. The CUDA version keeps the same chunk idea but adds two streams so host-device copies and kernel work can be queued independently. This is one of the first moments where CUDA stops feeling like *just C with funny angle brackets* and starts feeling like a real heterogeneous workflow.

Back to the outline page

## 8.2 Extension 2: Memory Coalescing

Coalescing deserves respect. The simple rule is:

> If neighboring threads usually access neighboring addresses, life is better.

That is not the full hardware story, but it is the right first story. If thread 0 goes here and thread 1 suddenly wanders off to another zip code, bandwidth tends to get grumpy.

**Files for this extension**

- C version: `code/c/09_memory_coalescing_cpu.c`
- CUDA version: `code/cuda/09_memory_coalescing_gpu.cu`

### 8.2.1 C version

Listing 23: Filename: `code/c/09_memory_coalescing_cpu.c` — C version: compare contiguous and strided access on the CPU

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static double seconds_now(void)
{
    return (double)clock() / (double)CLOCKS_PER_SEC;
}

int main(void)
{
    const int logical_n = 1 << 20;
    const int stride = 32;
    const int repeat = 20;
    const size_t input_n = (size_t)logical_n * (size_t)stride;
```

```

    float *in = (float *)malloc(input_n * sizeof(float));
    float *out = (float *)malloc((size_t)logical_n * sizeof(float));
    double t0, t1, t2, t3;

    if (!in || !out) {
        fprintf(stderr, "CPU allocation failed\n");
        free(in);
        free(out);
        return EXIT_FAILURE;
    }

    for (size_t i = 0; i < input_n; ++i) {
        in[i] = (float)(i % 100);
    }

    for (int i = 0; i < logical_n; ++i) {
        out[i] = in[i] + 1.0f;
    }
    for (int i = 0; i < logical_n; ++i) {
        out[i] = in[(size_t)i * (size_t)stride] + 1.0f;
    }

    t0 = seconds_now();
    for (int r = 0; r < repeat; ++r) {
        for (int i = 0; i < logical_n; ++i) {
            out[i] = in[i] + 1.0f;
        }
    }
    t1 = seconds_now();

    t2 = seconds_now();
    for (int r = 0; r < repeat; ++r) {
        for (int i = 0; i < logical_n; ++i) {
            out[i] = in[(size_t)i * (size_t)stride] + 1.0f;
        }
    }
    t3 = seconds_now();

    printf("CPU contiguous loop : %.6f s total, %.9f s per pass\n",
          t1 - t0, (t1 - t0) / repeat);
    printf("CPU strided loop : %.6f s total, %.9f s per pass\n",
          t3 - t2, (t3 - t2) / repeat);
    printf("Sample value : %.1f\n", out[7]);

    free(in);
    free(out);
    return EXIT_SUCCESS;
}
```

### 8.2.2 CUDA version

Listing 24: Filename: `code/cuda/09_memory_coalescing_gpu.cu` — CUDA version: compare contiguous and strided reads on the GPU

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <cuda_runtime.h>

#define CHECK_CUDA(call) \
    do { \
        cudaError_t err = (call); \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
                    cudaGetErrorString(err)); \
            return EXIT_FAILURE; \
        } \
    } while (0)

__global__ void contiguous_read_kernel(float *out, const float *in, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = in[i] + 1.0f;
    }
}

__global__ void strided_read_kernel(float *out, const float *in, int n, int stride)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        size_t idx = (size_t)i * (size_t)stride;
        out[i] = in[idx] + 1.0f;
    }
}

int main(void)
{
    const int logical_n = 1 << 20; /* number of output elements */
    const int stride = 32; /* spaced access distance */
    const int repeat = 100; /* repeat for stable timing */
    const size_t input_n = (size_t)logical_n * (size_t)stride;

    const int threads = 256;
    const int blocks = (logical_n + threads - 1) / threads;

    float *h_in = NULL;
    float *h_out = NULL;
    float *d_in = NULL;
    float *d_out = NULL;

    cudaEvent_t start, stop;
    float ms_contiguous = 0.0f;
    float ms_strided = 0.0f;

    h_in = (float *)malloc(input_n * sizeof(float));
    h_out = (float *)malloc((size_t)logical_n * sizeof(float));

    if (!h_in || !h_out) {
        fprintf(stderr, "Host allocation failed\n");
        free(h_in);
        free(h_out);
        return EXIT_FAILURE;
    }

```

```
    for (size_t i = 0; i < input_n; ++i) {
        h_in[i] = (float)(i % 100);
    }

    CHECK_CUDA(cudaMalloc((void **)&d_in, input_n * sizeof(float)));
    CHECK_CUDA(cudaMalloc((void **)&d_out, (size_t)logical_n * sizeof(float)));
    CHECK_CUDA(cudaMemcpy(d_in, h_in, input_n * sizeof(float), cudaMemcpyHostToDevice)
        );

    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    /* warm-up */
    contiguous_read_kernel<<<blocks, threads>>>(d_out, d_in, logical_n);
    strided_read_kernel<<<blocks, threads>>>(d_out, d_in, logical_n, stride);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    CHECK_CUDA(cudaEventRecord(start));
    for (int r = 0; r < repeat; ++r) {
        contiguous_read_kernel<<<blocks, threads>>>(d_out, d_in, logical_n);
    }
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    CHECK_CUDA(cudaEventElapsedTime(&ms_contiguous, start, stop));

    CHECK_CUDA(cudaEventRecord(start));
    for (int r = 0; r < repeat; ++r) {
        strided_read_kernel<<<blocks, threads>>>(d_out, d_in, logical_n, stride);
    }
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    CHECK_CUDA(cudaEventElapsedTime(&ms_strided, start, stop));

    CHECK_CUDA(cudaMemcpy(h_out, d_out, (size_t)logical_n * sizeof(float),
        cudaMemcpyDeviceToHost));

    printf("GPU contiguous read : %.3f ms total, %.6f ms per launch\n",
          ms_contiguous, ms_contiguous / repeat);
    printf("GPU strided read : %.3f ms total, %.6f ms per launch\n",
          ms_strided, ms_strided / repeat);
    printf("Sample value : %.1f\n", h_out[7]);

    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    CHECK_CUDA(cudaFree(d_in));
    CHECK_CUDA(cudaFree(d_out));
    free(h_in);
    free(h_out);

    return EXIT_SUCCESS;
}
```

The paired point here is easy to see. The C code already prefers locality, because caches are not decorative either. The CUDA code shows the same idea in a more visible way: the contiguous kernel lets adjacent threads read adjacent locations, while the strided kernel forces

them apart. Same machine, same math style, very different traffic pattern.

Back to the outline page

## 8.3 Extension 3: Error Handling as a Real Habit

Beginners often treat error handling like garnish. It is not garnish. It is the difference between finding the bug in two minutes and staring at nonsense results for half a day.

**Files for this extension**

- C version: `code/c/10_error_checking_cpu.c`
- CUDA version: `code/cuda/10_error_checking_gpu.cu`

### 8.3.1 C version

Listing 25: Filename: `code/c/10_error_checking_cpu.c` — C version: check allocation and input conditions

```c
#include <stdio.h>
#include <stdlib.h>

#define CHECK_ALLOC(ptr) \
    do { \
        if ((ptr) == NULL) { \
            fprintf(stderr, "Allocation failed at %s:%d\n", __FILE__, __LINE__); \
            return EXIT_FAILURE; \
        } \
    } while (0)

static int safe_scale(float *data, int n, float a)
{
    if (data == NULL) {
        fprintf(stderr, "safe_scale: null pointer\n");
        return -1;
    }
    if (n <= 0) {
        fprintf(stderr, "safe_scale: invalid length %d\n", n);
        return -1;
    }

    for (int i = 0; i < n; ++i) {
        data[i] *= a;
    }
    return 0;
}

int main(void)
{
    const int N = 8;
    float *data = (float *)malloc((size_t)N * sizeof(float));
    CHECK_ALLOC(data);

    for (int i = 0; i < N; ++i) {
        data[i] = (float)(i + 1);
    }
```

```

    if (safe_scale(data, N, 2.0f) != 0) {
        free(data);
        return EXIT_FAILURE;
    }

    printf("CPU checked result: ");
    for (int i = 0; i < N; ++i) {
        printf("%.1f ", data[i]);
    }
    printf("\n");

    free(data);
    return EXIT_SUCCESS;
}
```

### 8.3.2 CUDA version

Listing 26: Filename: `code/cuda/10_error_checking_gpu.cu` — CUDA version: check runtime calls, kernel launch status, and kernel completion

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) \
    do { \
        cudaError_t err = (call); \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
                    cudaGetErrorString(err)); \
            return EXIT_FAILURE; \
        } \
    } while (0)

__global__ void scale_kernel(float *data, float a, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        data[i] *= a;
    }
}

int main(void)
{
    const int N = 8;
    float h_data[N];
    float *d_data = NULL;

    for (int i = 0; i < N; ++i) {
        h_data[i] = (float)(i + 1);
    }

    CHECK_CUDA(cudaMalloc((void **)&d_data, (size_t)N * sizeof(float)));
    CHECK_CUDA(cudaMemcpy(d_data, h_data, (size_t)N * sizeof(float),
        cudaMemcpyHostToDevice));

```

```
    int threads = 128;
    int blocks = (N + threads - 1) / threads;
    scale_kernel<<<blocks, threads>>>(d_data, 2.0f, N);

    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());
    CHECK_CUDA(cudaMemcpy(h_data, d_data, (size_t)N * sizeof(float),
        cudaMemcpyDeviceToHost));

    printf("CUDA checked result: ");
    for (int i = 0; i < N; ++i) {
        printf("%.1f ", h_data[i]);
    }
    printf("\n");

    cudaFree(d_data);
    return EXIT_SUCCESS;
}
```

The CUDA pattern should be memorized early:

1. check every CUDA runtime call,
2. check the kernel launch with `cudaGetLastError()`,
3. synchronize when needed so runtime failures actually show up.

If you skip step 2 or step 3, CUDA can fail politely and quietly, which is very professional of it, but not especially helpful to the student.

Back to the outline page

## 9. Conclusion

This note is built around one honest rule: a beginner does not need the whole CUDA universe on day one. A beginner needs a small set of correct ideas explained clearly. That is why **every major program here is paired with a CPU-side C version and a GPU-side CUDA version**.

That pairing is not cosmetic. It lets the reader see, almost line by line, what actually changes:

- the serial loop becomes per-thread work,
- array indices become global thread indices,
- ordinary function calls become kernel launches,
- ordinary memory access becomes host-device dataflow,
- local reuse can become shared-memory cooperation.

That is still one of the most reliable ways to teach CUDA, especially to readers coming from a C background who want to see *exactly what changes and exactly what stays the same*, without extra fireworks.